

Complete CTE (Common Table Expression) Guide in MySQL 8+

A **CTE (Common Table Expression)** is a temporary result set that exists only during the execution of a query.

Syntax:

```
WITH cte_name AS  
(  
    SELECT ...  
)  
SELECT *  
FROM cte_name;
```

1. Create Sample Tables

Employees Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10,2),  
    manager_id INT,  
    joining_date DATE  
);
```

Insert Data

```
INSERT INTO employees VALUES  
(101,'Arun Kumar','IT',85000,NULL,'2020-01-15'),  
(102,'Priya Sharma','IT',75000,101,'2021-03-10'),  
(103,'Rahul Verma','HR',65000,101,'2021-07-12'),  
(104,'Sneha Reddy','Finance',95000,101,'2019-09-20'),  
(105,'Vikram Singh','Finance',70000,104,'2022-01-25'),  
(106,'Anjali Nair','IT',60000,102,'2023-04-01'),  
(107,'Karthik Rao','HR',55000,103,'2023-05-15'),
```

```
(108,'Meera Iyer','Sales',50000,101,'2022-08-01'),  
(109,'Rohit Gupta','Sales',52000,108,'2023-02-10'),  
(110,'Neha Joshi','IT',80000,102,'2021-11-05');
```

Why Use CTE?

Without CTE

```
SELECT *  
FROM  
(  
    SELECT department,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department  
) d;
```

With CTE

```
WITH dept_avg AS  
(  
    SELECT department,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department  
)  
SELECT *  
FROM dept_avg;
```

Much cleaner and easier to read.

2. Simple CTE

Find employees earning above average salary

```
WITH avg_sal AS  
(  
    SELECT AVG(salary) avg_salary  
    FROM employees
```

```
)  
SELECT e.*  
FROM employees e  
CROSS JOIN avg_sal a  
WHERE e.salary > a.avg_salary;
```

Output

Employee e	Salary
Arun	85000
Sneha	95000
Neha	80000

3. Multiple CTEs

Multiple CTEs are separated by commas.

```
WITH dept_avg AS  
(  
    SELECT department,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department  
)  
high_paid AS  
(  
    SELECT *  
    FROM employees  
    WHERE salary > 70000  
)  
SELECT h.emp_name,  
       h.salary,  
       d.avg_salary  
FROM high_paid h  
JOIN dept_avg d  
ON h.department = d.department;
```

4. CTE with JOIN

Find employees whose salary is above department average.

```
WITH dept_avg AS
(
    SELECT department,
           AVG(salary) avg_salary
    FROM employees
    GROUP BY department
)
SELECT e.emp_name,
       e.department,
       e.salary,
       d.avg_salary
FROM employees e
JOIN dept_avg d
ON e.department = d.department
WHERE e.salary > d.avg_salary;
```

5. CTE with Aggregate Functions

Department salary summary.

```
WITH dept_stats AS
(
    SELECT department,
           COUNT(*) total_employees,
           AVG(salary) avg_salary,
           MAX(salary) max_salary,
           MIN(salary) min_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_stats;
```

6. CTE with Window Functions

Rank employees by salary.

```
WITH emp_rank AS
(
    SELECT emp_name,
           department,
           salary,
           DENSE_RANK() OVER
           (
               PARTITION BY department
               ORDER BY salary DESC
           ) ranking
    FROM employees
)
SELECT *
FROM emp_rank;
```

7. CTE Referencing Another CTE

```
WITH dept_avg AS
(
    SELECT department,
           AVG(salary) avg_salary
    FROM employees
    GROUP BY department
),
above_avg AS
(
    SELECT e.*
    FROM employees e
    JOIN dept_avg d
    ON e.department = d.department
    WHERE e.salary > d.avg_salary
)
SELECT *
FROM above_avg;
```

8. CTE with CASE Statement

Salary Classification.

```
WITH salary_band AS
(
    SELECT emp_name,
           salary,
           CASE
               WHEN salary >= 90000 THEN 'High'
               WHEN salary >= 70000 THEN 'Medium'
               ELSE 'Low'
           END band
    FROM employees
)
SELECT *
FROM salary_band;
```

9. CTE with Subquery

Highest paid employee from each department.

```
WITH dept_max AS
(
    SELECT department,
           MAX(salary) max_salary
    FROM employees
    GROUP BY department
)
SELECT e.*
FROM employees e
JOIN dept_max d
ON e.department = d.department
AND e.salary = d.max_salary;
```

10. Recursive CTE

Most important CTE type.

Used for:

- Hierarchies
- Organizational Charts
- Bill of Materials

- Parent Child Relationships
 - Tree Traversal
-

Employee Hierarchy

WITH RECURSIVE emp_hierarchy AS

(

-- Anchor Query

SELECT

emp_id,
emp_name,
manager_id,
1 AS level

FROM employees

WHERE manager_id IS NULL

UNION ALL

-- Recursive Query

SELECT

e.emp_id,
e.emp_name,
e.manager_id,
h.level + 1

FROM employees e

JOIN emp_hierarchy h

ON e.manager_id = h.emp_id

)

SELECT *

FROM emp_hierarchy;

Output

Arun Kumar	Level 1
Priya Sharma	Level 2
Rahul Verma	Level 2
Sneha Reddy	Level 2
Meera Iyer	Level 2
Anjali Nair	Level 3
Neha Joshi	Level 3
Karthik Rao	Level 3

11. Recursive CTE with Path

Display reporting chain.

```
WITH RECURSIVE emp_tree AS
(
    SELECT
        emp_id,
        emp_name,
        manager_id,
        CAST(emp_name AS CHAR(500)) path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        CONCAT(t.path, ' -> ', e.emp_name)
    FROM employees e
    JOIN emp_tree t
    ON e.manager_id = t.emp_id
)
SELECT *
FROM emp_tree;
```

Example Output

```
Arun Kumar
Arun Kumar -> Priya Sharma
Arun Kumar -> Priya Sharma -> Anjali Nair
```

12. Recursive Number Generator

Generate numbers from 1 to 10.


```
WITH RECURSIVE numbers AS
(
    SELECT 1 AS num

    UNION ALL

    SELECT num + 1
    FROM numbers
    WHERE num < 10
)
SELECT *
FROM numbers;
```

13. Recursive Date Generator

Generate dates.

```
WITH RECURSIVE calendar AS
(
    SELECT DATE('2025-01-01') dt

    UNION ALL

    SELECT dt + INTERVAL 1 DAY
    FROM calendar
    WHERE dt < '2025-01-10'
)
SELECT *
FROM calendar;
```

14. CTE with DELETE

Delete employees earning below average salary.

```
WITH avg_sal AS
(
    SELECT AVG(salary) avg_salary
    FROM employees
)
```

```
DELETE FROM employees
WHERE salary <
(
  SELECT avg_salary
  FROM avg_sal
);
```

15. CTE with UPDATE

```
WITH dept_avg AS
(
  SELECT department,
         AVG(salary) avg_salary
  FROM employees
  GROUP BY department
)
UPDATE employees e
JOIN dept_avg d
ON e.department = d.department
SET e.salary = e.salary * 1.10
WHERE e.salary < d.avg_salary;
```

16. Top N Employees Per Department

Common Interview Question

```
WITH ranked_emp AS
(
  SELECT *,
         ROW_NUMBER() OVER
         (
           PARTITION BY department
           ORDER BY salary DESC
         ) rn
  FROM employees
)
SELECT *
FROM ranked_emp
WHERE rn <= 2;
```

17. Find Duplicate Records using CTE

```
WITH duplicates AS
(
    SELECT emp_name,
           COUNT(*) cnt
    FROM employees
    GROUP BY emp_name
    HAVING COUNT(*) > 1
)
SELECT *
FROM duplicates;
```

18. Running Total using CTE

```
WITH salary_running AS
(
    SELECT emp_name,
           salary,
           SUM(salary) OVER
           (
               ORDER BY salary
           ) running_total
    FROM employees
)
SELECT *
FROM salary_running;
```

19. Gap Analysis (Missing IDs)

```
WITH RECURSIVE nums AS
(
    SELECT 101 AS id

    UNION ALL

    SELECT id + 1
    FROM nums
    WHERE id < 110
```

```
)  
SELECT n.id  
FROM nums n  
LEFT JOIN employees e  
ON n.id = e.emp_id  
WHERE e.emp_id IS NULL;
```

20. Interview-Worthy CTE Scenarios

Scenario	CTE Type
Department Average Salary	Simple CTE
Above Department Average	Multiple CTE
Employee Ranking	CTE + Window Function
Top N Employees	CTE + ROW_NUMBER
Employee Hierarchy	Recursive CTE
Generate Calendar Dates	Recursive CTE
Running Totals	CTE + Window Function
Find Missing IDs	Recursive CTE
Detect Duplicates	CTE + Aggregation
Organizational Chart	Recursive CTE
Data Cleansing	CTE + DELETE
Mass Updates	CTE + UPDATE

CTE Interview Definition

A Common Table Expression (CTE) is a temporary named result set created using the **WITH** clause that improves query readability, modularity, and maintainability. It can be referenced multiple times within the same query and supports recursive processing for hierarchical data.

Types of CTEs

1. Simple CTE
2. Multiple CTE
3. Nested/Dependent CTE
4. Recursive CTE
5. CTE with JOIN
6. CTE with Aggregation
7. CTE with Window Functions
8. CTE with DML (INSERT/UPDATE/DELETE)
9. CTE for Hierarchies
10. CTE for Sequence/Date Generation

These cover virtually all CTE patterns you'll encounter in real-world Data Engineering, SQL development, ETL, and interview scenarios.